

Komite - despliegue en Amazon Lightsail

Esta guía sigue el mismo patrón usado en Vallestelar Sentinel:

```
Cloudflare DNS + SSL
-> Amazon Lightsail Ubuntu
-> Nginx reverse proxy
-> Docker Compose
  - web Nuxt, puerto interno 3000
  - api FastAPI + backoffice, puerto interno 8000
  - PostgreSQL
  - Redis
```

URLs propuestas

```
Web Nuxt:      https://komite.cl
API Swagger:   https://api.komite.cl/docs
Backoffice:    https://api.komite.cl/login
```

1. Subir el repo a GitHub

Desde Windows, en la carpeta del proyecto:

```
cd C:\Users\anton\OneDrive\Escritorio\REPO_VALLESTELAR\KOMITE
git init
git add .
git commit -m "first deploy setup"
git branch -M main
git remote add origin https://github.com/vallestelar/komite.git
git push -u origin main
```

2. Preparar DNS en Cloudflare

Crea estos registros A, apuntando a la IP pública de Lightsail:

```
@      -> IP_PUBLICA_LIGHTSAIL
www     -> IP_PUBLICA_LIGHTSAIL
api     -> IP_PUBLICA_LIGHTSAIL
```

En Cloudflare:

```
SSL/TLS -> Overview -> Full  
Always Use HTTPS -> On  
Automatic HTTPS Rewrites -> On
```

3. Conectarse por SSH a Lightsail

```
ssh -i "C:\Users\anton\Downloads\LightsailDefaultKey-us-east-1.pem"  
ubuntu@44.203.17.117
```

Cambia la IP si vas a usar otra instancia.

4. Instalar dependencias del servidor

```
sudo apt update  
sudo apt install -y git nginx certbot python3-certbot-nginx ca-certificates curl
```

Instalar Docker:

```
curl -fsSL https://get.docker.com | sudo sh  
sudo usermod -aG docker ubuntu
```

Sal y vuelve a entrar por SSH para que el grupo `docker` aplique.

5. Clonar el proyecto

```
sudo mkdir -p /opt/komite  
sudo chown ubuntu:ubuntu /opt/komite  
git clone https://github.com/vallestelar/komite.git /opt/komite  
cd /opt/komite
```

6. Crear el archivo `.env` real

```
cp .env.production.example .env  
nano .env
```

Cambia como minimo:

```
POSTGRES_PASSWORD=una_password_fuerte
JWT_SECRET_KEY=un_secreto_largo_y_aleatorio
SEED_ADMIN_PASSWORD=una_password_admin_fuerte
Nuxt_PUBLIC_API_BASE=https://api.komite.cl
```

El `.env` real no se sube a GitHub.

7. Levantar contenedores

```
docker compose up -d --build
docker compose ps
```

Aplicar migraciones:

```
docker compose exec -T api aerich upgrade
```

Crear roles y usuario admin inicial:

```
docker compose exec api python scripts/seed_auth.py
```

8. Configurar Nginx

Frontend:

```
sudo nano /etc/nginx/sites-available/komite
```

```
server {
    listen 80;
    server_name komite.cl www.komite.cl;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

API y backoffice:

```
sudo nano /etc/nginx/sites-available/api-komite
```

```
server {  
    listen 80;  
    server_name api.komite.cl;  
  
    location / {  
        proxy_pass http://127.0.0.1:8000;  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

Activar sitios:

```
sudo ln -s /etc/nginx/sites-available/komite /etc/nginx/sites-enabled/  
sudo ln -s /etc/nginx/sites-available/api-komite /etc/nginx/sites-enabled/  
sudo rm -f /etc/nginx/sites-enabled/default  
sudo nginx -t  
sudo systemctl reload nginx
```

9. Certificados HTTPS

```
sudo certbot --nginx -d komite.cl -d www.komite.cl -d api.komite.cl  
sudo certbot certificates  
sudo systemctl status certbot.timer
```

10. Probar

```
curl http://127.0.0.1:8000/health  
curl http://127.0.0.1:3000
```

En navegador:

```
https://komite.cl  
https://api.komite.cl/docs
```

```
https://api.komite.cl/login
```

11. Deploy diario

En local:

```
git add .  
git commit -m "descripcion del cambio"  
git push origin main
```

En el servidor:

```
cd /opt/komite  
chmod +x deploy.sh  
./deploy.sh
```

12. Logs y diagnostico

```
docker compose ps  
docker compose logs -f api  
docker compose logs -f web  
docker compose logs -f postgres  
docker compose logs -f redis
```

13. Acceso PostgreSQL por tunel SSH

Desde Windows:

```
ssh -i "C:\Users\anton\Downloads\LightsailDefaultKey-us-east-1.pem" -L  
5433:localhost:5432 ubuntu@44.203.17.117
```

En pgAdmin:

```
Host: localhost  
Port: 5433  
Maintenance database: komite_db  
Username: postgres  
Password: valor de POSTGRES_PASSWORD del .env
```

14. Backup PostgreSQL

Crear backup en el servidor:

```
docker exec komite_postgres pg_dump -U postgres -d komite_db -Fc -f  
/tmp/komite_db.backup  
docker cp komite_postgres:/tmp/komite_db.backup /home/ubuntu/komite_db.backup
```

Descargar a Windows:

```
scp -i "C:\Users\anton\Downloads\LightsailDefaultKey-us-east-1.pem"  
ubuntu@44.203.17.117:/home/ubuntu/komite_db.backup "C:\Users\anton\Desktop\"
```

15. No ejecutar

No ejecutes estos comandos salvo que quieras borrar datos:

```
docker compose down -v  
docker volume prune  
docker system prune --volumes  
docker volume rm komite_postgres_data
```

La base de datos vive en el volumen Docker `komite_postgres_data`.